



UNIT-2

STRING MATCHING AND COMPUTATIONAL GEOMETRY

INTRODUCTION TO STRING MATCHING

- Find pattern occurrences within a text.
- Fundamental problem with many applications.
- **Components of String Matching:**
 - Pattern: The string to search for. Example: "abc"
 - Text: The larger string where the pattern is searched.
 - Example: "abccabccabc"

TYPES OF STRING MATCHING:

- **Exact Matching:**

- The goal is to find all exact occurrences of the pattern in the text.
- Example:Text: "abababa"
- Pattern: "aba"
- Output: Positions 0, 2, and 4 (0-based index).

- **Approximate Matching:**

- The pattern can match the text approximately, allowing mismatches or edits.
- Example: DNA sequence alignment, where "AGCT" may match "AGTT" with one substitution.

APPLICATIONS OF STRING MATCHING



CHALLENGES IN STRING MATCHING:

- **Efficiency:** For long texts or patterns, naive approaches are too slow.
- **Variability:** In approximate matching, handling mismatches increases complexity.

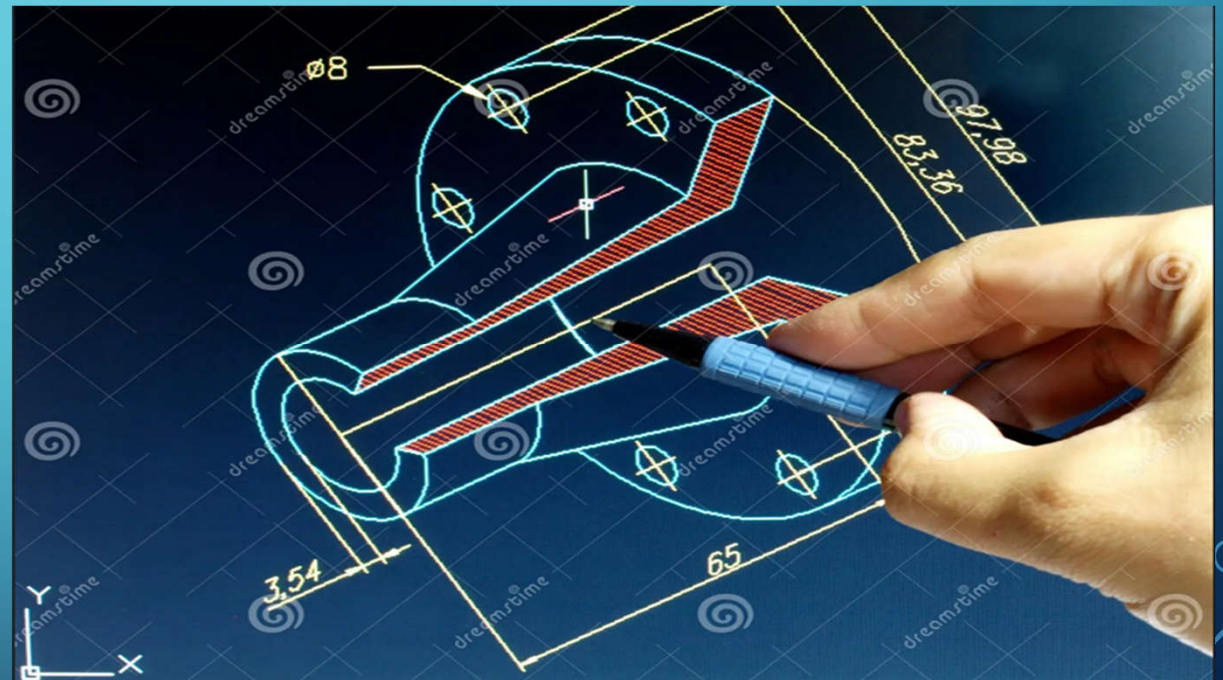
COMPUTATIONAL GEOMETRY

- **Introduction to Computational Geometry**
- Definition: Algorithms to solve geometric problems involving points, lines, polygons, etc.
- **Applications:**
 - Computer graphics
 - Robotics and pathfinding
 - Geographic Information Systems (GIS)
 - Game development

KEY CHARACTERISTICS

- Focuses on the design, analysis, and implementation of geometric algorithms.
- Deals with problems in two or more dimensions.

APPLICATIONS



SEQUENTIAL SEARCH (LINEAR SEARCH)

- Sequentially check elements in list.
- Find desired key or traverse all.
- Simple algorithm for searching arrays.
- Works on sorted or unsorted data.
- Compares each element with key.
- Stops if key is found early. Continues until key or end.
- Inefficient for large data sizes.

STEPS OF SEQUENTIAL SEARCH

- Start from the first element in the list.
- Compare the key with the current element.
- If a match is found, return the index of the element.
- If no match is found, move to the next element.
- Repeat until the element is found or the list ends.

PSEUDOCODE

- **SequentialSearch(array, key):**
 - **for i = 0 to array.length - 1:**
 - **if array[i] == key:**
 - **return i # Return index of key**
 - **return -1 # Key not found**

- **Advantages:**
- Simple to implement.
- Works on both sorted and unsorted data.
- No additional memory required.
- **Disadvantages:**
- Inefficient for large datasets.
- Requires linear traversal, making it slower compared to other search algorithms like binary search.

APPLICATIONS:

- Small datasets where simplicity is preferred.
- Searching in unsorted or unordered lists.
- Scenarios where preprocessing (like sorting) is not feasible.

BRUTE-FORCE STRING MATCHING (NAÏVE STRING MATCHING)

- Simplest method for string matching.
- Finds a pattern within a text.
- Checks every position for a match.
- Compares pattern characters with text.
- Works without preprocessing text or pattern.
- Inefficient for large text or patterns.

Brute-Force String Matching



- pattern: a string of m characters to search for
- text: a (longer) string of n characters to search in
- problem: find a substring in the text that matches the pattern

Brute-force algorithm

Step 1 Align pattern at beginning of text

Step 2 Moving from left to right, compare each character of pattern to the corresponding character in text until

- all characters are found to match (successful search); or
- a mismatch is detected

Step 3 While pattern is not found and the text is not yet exhausted, realign pattern one position to the right and repeat Step 2

Examples of Brute-Force String Matching



1. Pattern: 001011

Text: 10010101101001100101111010

2. Pattern: happy

Text: It is never too late to have a
happy childhood.

Pseudocode and Efficiency



ALGORITHM *BruteForceStringMatch*($T[0..n - 1]$, $P[0..m - 1]$)

//Implements brute-force string matching

//Input: An array $T[0..n - 1]$ of n characters representing a text and

// an array $P[0..m - 1]$ of m characters representing a pattern

//Output: The index of the first character in the text that starts a

// matching substring or -1 if the search is unsuccessful

for $i \leftarrow 0$ **to** $n - m$ **do**

$j \leftarrow 0$

while $j < m$ **and** $P[j] = T[i + j]$ **do**

$j \leftarrow j + 1$

if $j = m$ **return** i

return -1

Time efficiency:

Advantages:

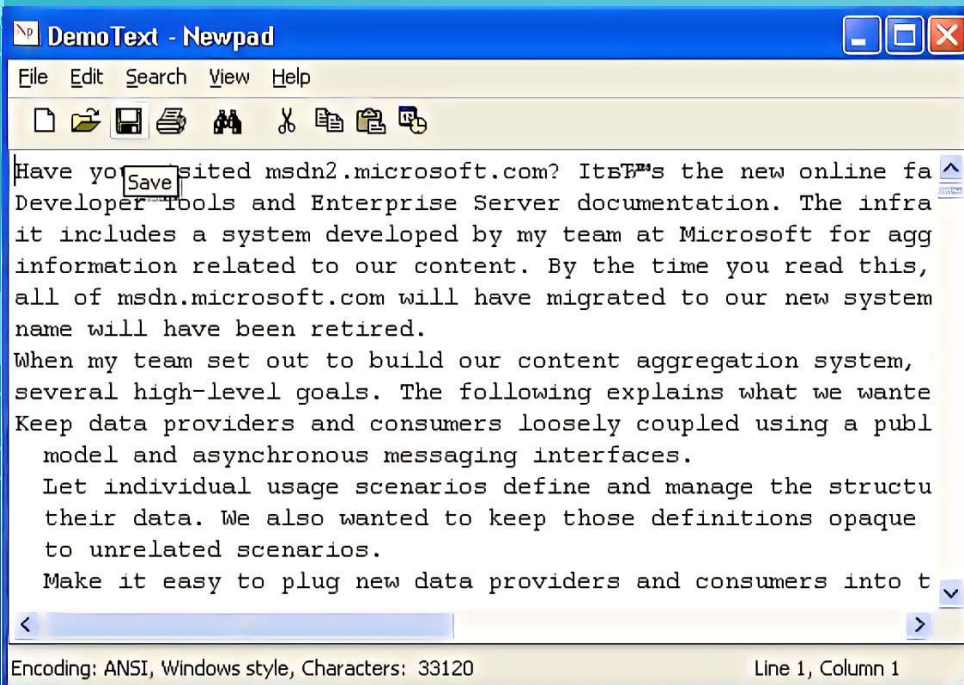
- Simple and easy to implement.
- Does not require preprocessing of the text or pattern.

Disadvantages:

- Inefficient for large texts and patterns.
- High time complexity compared to optimized algorithms like KMP or Boyer-Moore.

- Example:
- Text: "abcabcabc"
- Pattern: "abc"
- Result: Pattern found at positions 0, 3, and 6.

APPLICATION



COMP3456 – adapted from textbook slides

www.bioalgorithms.info

Exact Pattern Matching: An Example

- *PatternMatching* algorithm for:

- Pattern GCAT

- Text CGCATC

GCAT
CGCATC

GCAT
CGCATC

GCAT
CGCATC

GCAT
CGCATC

GCAT
CGCATC